

Murdoch University

ICT 283 Data Structures & Abstraction

PTC – TMA 2026

34528531 Muammar Shabir Bin Mohd Bashir

Assignment 2

Table of Contents

Overview	3
Data Structure Design	3
Architecture Pattern (MVC).....	4
Implementation Details & Techniques.....	4
Advantages & Disadvantages	5
Conclusion	6
Github	6
Test Plan	6

Overview

The software is an interactive, command-line meteorological data analysis program designed to load and process historical weather records from multiple data files. In accordance with the project specifications, the application provides a user-friendly console menu that allows users to execute targeted temporal analyses without graphical user interface (GUI) elements. The program calculates average wind speed and sample standard deviation for specific months, derives ambient air temperature statistics across full years, computes sample Pearson Correlation Coefficients (sPCC) for wind, temperature, and solar radiation across all loaded years, and exports monthly statistical summaries, including Mean Absolute Deviation (MAD), directly to an external report named WindTempSolar.csv.

Data Structure Design

To fulfill the mandatory assignment requirement of utilizing both a template Binary Search Tree (BST) with function pointers and a map data structure, the application implements a four-tier nested hierarchy encapsulated within the DataDB class:

DataDB

- └ Bst<YearData> (Chronological BST of Years)
 - └ Bst<MonthData> (Numerical BST of Months 1-12)
 - └ DayMap [std::map<int, WeatherData>] (Ordered Map of Days 1-31)
 - └ Vector<WeatherEntry> (Encapsulated Dynamic Array of Intraday Records)

Rationale and Theoretical Justification

- **Bst<YearData> and Bst<MonthData>:** A Binary Search Tree is chosen for the top two tiers because it provides $O(\log n)$ search, insertion, and traversal complexity without requiring contiguous memory allocation. Using a BST prevents the spatial waste of pre-allocating empty multi-dimensional arrays for non-existent periods.
- **DayMap (std::map<int, WeatherData>):** The Standard Template Library (STL) std::map is deployed at the day level.
- **Vector<WeatherEntry>:** For temporary use to do calculations

Architecture Pattern (MVC)

The application strictly enforces separation of concerns by organizing components into the Model-View-Controller (MVC) architectural pattern, ensuring that data classes perform zero input/output (I/O) operations and math routines remain decoupled from weather data types:

- **The Model:** Represents data persistence, state, and business logic. It consists of the domain entities WeatherEntry, Date, Time, and the nested container DataDB. All statistical calculations are isolated within the static Calculate class, ensuring that formulas for standard deviation, average, total, MAD, and sPCC operate independently of UI presentation or file storage formats.
- **The View:** Manages user interaction and output presentation. This layer is implemented through overloaded stream insertion operators (<<), locale-specific formatting in DateFormatter, console text formatting in Menu::DisplayMenu, and CSV file structuring in Menu::outputSummary.
- **The Controller:** Implemented by the Menu class. The controller mediates between the user and the system by parsing CSV source files (LoadFromSourceFile and LoadRecords), querying the DataDB model for specific temporal records, feeding extracted datasets into the Calculate engine, and routing the resulting metrics to console or file outputs.

Implementation Details & Techniques

Decoupled BST Traversals via Function Pointers

To comply with the assignment constraint requiring function pointers while strictly forbidding C++11 lambda functions, std::function, or range-based for loops, the custom Bst<T> class implements context-pointer traversal overloads:

```
template <class T>
void Bst<T>::InOrderTraversal(void (*process)(T&, void*), void* userData);
```

During Menu Option 3 (sPCC calculation across all years for a specified month), the controller injects a standalone static callback function, sPCC_Visit_Func, along with a pointer to an SPCC_Collector structure into DataDB::TraverseYears. As the binary tree traverses node-by-node, the function pointer executes the callback, extracting wind speed, temperature, and solar radiation across all historical years into linear vectors. This technique maintains class modularity by keeping the BST data structure entirely agnostic to application-specific calculations.

Decoupled Statistical Engine and Linear Bridging

Because mathematical formulas like Pearson Correlation and standard deviation require iterating over sequential datasets, the controller implements linear bridging methods such as `getRecordsForMonthAndYear`. This method queries the hierarchical `DataDB` tree, retrieves the target `DayMap`, and flattens the nested intraday records into a sequential `WeatherData` (`Vector<WeatherEntry>`) container. The static `Calculate` engine then processes these linear vectors using generic string column identifiers ("S", "T", "SR"), preventing mathematical coupling to the weather data classes.

Defensive Programming and Exception Handling

To prevent system crashes when processing corrupted or non-numeric CSV source files, `Menu::LoadRecords` tokenizes data strings using `std::stringstream` within a structured try-catch block. If string-to-float or string-to-integer conversions (`stoi`, `stof`) trigger `std::invalid_argument` or `std::out_of_range` exceptions, the controller intercepts the error, outputs a console warning, and safely skips the malformed record without halting program execution.

Advantages & Disadvantages

Advantages

- **Strict Specification Adherence:** The design fulfills all mandatory course constraints by successfully integrating a template BST with function pointers, an STL map, and an encapsulated custom `Vector` wrapper.
- **Optimal Search Efficiency:** Structuring years and months into binary search trees allow the application to locate specific monthly records without scanning entire multi-year data files linearly.
- **High Modularity and Reusability:** Isolating statistical logic inside `Calculate` and prohibiting I/O operations within domain data models ensures that mathematical routines and data structures can be reused across different C++ applications without modification.

Disadvantages

- **Increased Spatial Overhead:** Dynamic tree node allocations introduce extra memory consumption because every `YearData` and `MonthData` node must store two child pointers (`m_left` and `m_right`).
- **Memory Duplication During Flattening:** Bridging the hierarchical storage model with the linear statistical engine requires methods like `getRecordsForMonthAndYear` to copy

WeatherEntry objects into temporary Vector containers, which increases CPU overhead and dynamic memory allocations during heavy reporting tasks.

- **Architectural and Traversal Complexity:** Relying on void context pointers (void* userData) and multi-level callback functions to extract data across nested trees introduces code complexity that is more challenging to debug and maintain compared to simple linear indexing.

Conclusion

By morphing a custom template BST, an STL map, and an encapsulated dynamic Vector within an MVC framework, the design successfully resolves the architectural tension between non-linear sparse data storage and sequential statistical computation. Despite minor spatial overhead incurred during data flattening, the software achieves complete separation of concerns, robust exception handling, and excellent algorithmic efficiency.

Github

<https://github.com/liquidrain-zip/2026-weather-analysis>

Test Plan

Test ID	Description & Rationale	Actual Test Data	Expected Output	Passed
LD-01	Valid Data Ingestion: Verifies that the application correctly reads data_source.txt, opens listed CSV files, parses date, time, and float fields, and populates the nested DataDB BST/Map hierarchy without data loss.	data/data_source.txt containing valid relative paths (e.g., data/2025_Weather.csv with 500 valid records).	Successfully loaded 500 weather records. (Main menu is displayed subsequent to loading).	✓
LD-02	Missing Source File Handling: Verifies defensive I/O handling when a data file listed in data_source.txt does not exist or cannot be opened.	data_source.txt listing non_existent_file.csv.	Error: Could not open file data/non_existent_file.csv. 0 records loaded.	✓

LD-03	Corrupted CSV Record Handling: Verifies that non-numeric characters or malformed floating-point values in data columns trigger exception handling (invalid_argument / out_of_range), skipping only the bad record without crashing.	A record containing text in numeric fields: 16/01/2025 09:00:00,ERR,25.5,150.0.	Warning: Skipped record due to non-numeric data fields. (Program continues loading remaining valid records).	✓
MN-01	Menu Input - Non-Numeric Character: Verifies that entering characters or symbols at the main menu triggers stream failure handling, clears error flags, and re-prompts the user.	User enters: abc.	Invalid input. Please enter a number from the menu. (Menu is re-displayed).	✓
MN-02	Menu Input - Out-of-Range Numeric: Verifies that numeric inputs outside the valid range (1 to 5) are intercepted by the default switch case.	User enters: 9.	Invalid choice. Please enter a number between 1 and 5.	✓
OP1-01	Option 1 - Valid Month & Year with Data: Verifies aggregation, average wind speed calculation (converting m/s to km/h), and sample standard deviation calculation for a specific temporal query.	Choice: 1, Month: 3 (January), Year: 2015	March 2015: Average speed: 20.3 km/h Sample stdev: 9.1	✓
OP1-02	Option 1 - Querying Period with No Data: Verifies that querying a valid month and year that lacks historical observations outputs the required assignment specification string.	Choice: 1, Month: 3 (March), Year: 1905.	March 1905: No Data.	✓
OP1-03	Option 1 - Invalid Month Boundary Check: Verifies that entering an out-of-bounds month index (less	Choice: 1, Month: 13.	Invalid month. Returning to menu.	✓

	than 1 or greater than 12) aborts the operation cleanly.			
OP2-01	Option 2 - Valid Year with Complete/Partial Data: Verifies month-by-month loop iteration, ambient air temperature averages, sample standard deviation calculations, and missing month formatting across an entire year.	Choice: 2, Year: 2015 (Data present only for March).	2021 January: No Data February: No Data March: average: 21.7 degrees C, stdev: 2.0 April: No Data May: No Data June: No Data July: No Data August: No Data September: No Data October: No Data November: No Data December: No Data	✓
OP2-02	Option 2 - Querying Year with No Data: Verifies that requesting temperature averages for a year entirely absent from the DataDB tree outputs "No Data" for all 12 months.	Choice: 2, Year: 1800.	1800 January: No Data February: No Data March: No Data April: No Data May: No Data June: No Data July: No Data August: No Data September: No Data October: No Data November: No Data December: No Data	✓
OP3-01	Option 3 - sPCC Calculation (Sufficient Data): Verifies the context-pointer BST traversal (TraverseYears), data collector aggregation across all loaded years for a target month, and sample Pearson Correlation Coefficient calculations for S_T, S_R, and T_R.	Choice: 3, Month: 3 (March) (Dataset containing 2 or more historical records across all years).	Sample Pearson Correlation Coefficient for March S_T: 0.38 S_R: 0.37 T_R: 0.62	✓
OP4-01	Option 4 - CSV File Summary Export (Valid Year): Verifies file creation (WindTempSolar.csv), comma-separated formatting, inclusion of Mean Absolute Deviation	Choice: 4, Year: 2021 (Data for March).	Console: Data for year 2021 written to WindTempSolar.csv. File Content (WindTempSolar.csv): 2021 March,20.3(4.5),21.7(2.0),32.6	✓

	(stdev, mad), solar radiation unit conversion (Wh/m2 divided by 1000 to kWh/m2), and strict omission of missing months.			
OP4-02	Option 4 - CSV Summary Export (Empty Year): Verifies adherence to the strict assignment rule: if an entire year's data is unavailable, output just the year on line 1 and "No Data" on line 2 of the CSV file.	Choice: 4, Year: 2026 (No records in database).	Console: No data found for year 2026. Output written to WindTempSolar.csv File Content (WindTempSolar.csv): 2026 No Data	✓
OP5-01	Option 5 - Application Exit: Verifies that selecting menu option 5 cleanly breaks the controller loop and terminates the program with return code 0.	Choice: 5.	Exiting program. (Program terminates cleanly).	✓